

CS 4150 ALGORITHMS

Spring 2026

Assignment 7: Graph Algorithms II

Due Date: 11:59 p.m., Thursday, April 16, 2026

Note: In this assignment, all graphs are represented by **adjacency lists**.

1. **(20 points)** Given a directed graph G with n vertices and m edges, each edge (u, v) has a weight $w(u, v)$, which can be positive, zero, or negative. The *bottleneck-weight* of a path in G is defined to be the **largest** weight of all edges in the path. Let s and t be two vertices of G . A *minimum bottleneck-weight path* from s to t is a path with the smallest bottleneck-weight among all paths from s to t in G . Refer to Figure 1 for an example.

Modify Dijkstra's algorithm to compute minimum bottleneck-weight paths from s to all other vertices of G . Your algorithm does not have to output all paths but only need to compute the correct predecessor information for all vertices (which forms a *minimum bottleneck-weight path tree* with s as the root), as we did in class for Dijkstra's algorithm. Your algorithm is required to have the same time complexity as Dijkstra's algorithm, i.e., $O((n + m) \log n)$ time.

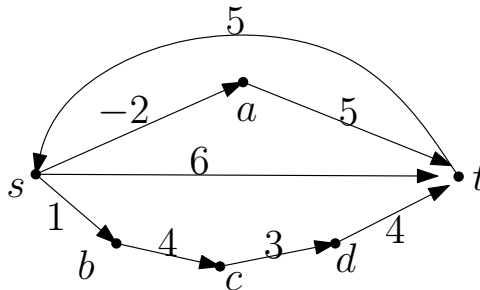


Figure 1: Here is a minimum bottleneck-weight path from s to t : (s, b, c, d, t) , whose bottleneck-weight is 4.

2. **(10 points)** Let G be a directed graph whose edges have weights. Some edge weights are negative but G does not have a negative cycle (i.e., the total weight of the edges of any cycle of G is nonnegative). Let s and t be two vertices of G . Suppose we want to find a shortest path from s to t in G . We know that Dijkstra's algorithm does not work for graphs with negative edge weights. We also know that Bellman-Ford algorithm can solve the problem in $O(n(n + m))$ time, where n is the number of vertices and m is the number of edges of the graph. Professor A instead proposes the following faster algorithm, which takes $O((n + m) \log n)$ time. The algorithm has three steps.
 - (a) First, find the smallest edge weight W in G (so W is a negative value because some edges of G have negative weights).
 - (b) Second, add the value $-W$ to the weight of every edge of G (note that because W is negative, $-W$ is a positive value). In this way, every edge weight now becomes non-negative. Let G' be the new graph with the new edge weights.

- (c) Third, because all edge weights of G' are nonnegative, we can apply Dijkstra's algorithm to find a shortest path $\pi(s, t)$ in G' .

Professor A claims that the path $\pi(s, t)$ found by the above algorithm is also a shortest path in the original graph G . Do you agree that this is correct? Please justify your answer (i.e., either explain why it is correct or give a counterexample showing that it is not correct).

3. **(20 points)** Let G be an undirected connected graph with n vertices and m edges. Suppose each edge of G has a color of either *blue* or *red*.

Design an algorithm to find a spanning tree T of G such that T has as **few red** edges as possible. Your algorithm should run in $O((n + m) \log n)$ time.

Total Points: 50